TAKE-HOME ASSIGNMENT

School Admission System

Assistant Software Engineer Recruitment

পদ **Assistant Software Engineer**	সময়সীমা **11 November 2026 (11:59:59 PM)**
STACK **.NET 8+ · PostgreSQL · React 18+ · Docker**	জমা দেবেন **নিজের public GitHub repo**

১ Assignment: School Admission System

১.১ পরিস্থিতি

একটা বেসরকারি school নতুন শিক্ষাবর্ষে online-এ ভর্তি নেবে। এই school-এ ভর্তি হয় "আগে আবেদন করলে আগে পাবে" ভিত্তিতে, lottery নেই। প্রতিটা class-এর প্রতিটা shift-এ seat সীমিত (যেমন "Class 6 – Morning Shift"-এ ১২০টা)। ভর্তি খোলার সাথে সাথে হাজার হাজার অভিভাবক একসাথে আবেদন করেন।

আবেদন করলে seat কিছু সময়ের জন্য শিক্ষার্থীর নামে **hold** থাকে। এই সময়ের মধ্যে ভর্তি ফি জমা দিলে ভর্তি **confirm** হয়। সময়ের মধ্যে ফি জমা না হলে seat আবার অন্যদের জন্য খুলে যায়।

আপনাকে এই ভর্তি system-এর backend আর React UI দুটোই বানাতে হবে। School admin আর অভিভাবক সব কাজ UI থেকেই করবেন।

> Brief-এ সব বিষয় স্পষ্ট করে বলা নেই। সেখানে আপনার কাছে যেটা যুক্তিসঙ্গত মনে হয়, সেটা ধরে নিয়ে কাজ করুন এবং `ASSUMPTIONS.md`-এ কারণসহ লিখুন।

১.২ Technology

- **Backend:** C# (.NET 8 বা নতুন) আর PostgreSQL। ORM আপনার পছন্দমতো।
- **Frontend:** React 18 বা নতুন, JavaScript বা TypeScript। Vite দিয়ে শুরু করার পরামর্শ রইল। UI library আপনার পছন্দমতো।
- Backend চলবে `http://localhost:8080`-এ। Database-এর connection environment variable দিয়ে বদলানো যেতে হবে।
- Frontend থেকে backend-এর URL environment variable দিয়ে বদলানো যেতে হবে।
- পুরো project Docker দিয়ে চলবে। Repository-র root-এ একটা `docker-compose.yml` রাখুন, যাতে শুধু `docker compose up` দিলেই PostgreSQL, backend আর frontend একসাথে চালু হয়। আমাদের machine-এ Docker ছাড়া আর কিছু install করতে হবে না। আমরা এভাবেই app চালাব।

- AI tool ব্যবহার করা যাবে।

১.৩ UI-তে যা থাকবে

একটা class-এ একাধিক shift থাকতে পারে। এই system-এ প্রতিটা shift আলাদা একটা "class" হিসেবে তৈরি হবে, নিজের আলাদা seat সংখ্যা নিয়ে। যেমন "Class 6 – Morning Shift" আর "Class 6 – Day Shift" দুটো আলাদা class।

Login লাগবে না। Page-এর উপরে একটা ঘরে শিক্ষার্থীর জন্ম নিবন্ধন নম্বর লেখা যাবে। UI এটাকেই শিক্ষার্থীর পরিচয় হিসেবে ব্যবহার করবে।

Screen	কাজ
Class তৈরি (school admin)	Class-এর নাম আর seat সংখ্যা দিয়ে নতুন class
Class তালিকা	প্রতিটা class-এর মোট, hold-এ থাকা, নিশ্চিত আর খালি seat; "আবেদন করুন" button
আবেদন	অবস্থা, মেয়াদ শেষ হতে কত সময় বাকি, "ভর্তি ফি দিন" আর "বাতিল" button
আমার আবেদন	এই শিক্ষার্থীর সব আবেদন আর প্রতিটার অবস্থা

"ভর্তি ফি দিন" চাপলে ধরে নেবেন payment সফল হয়েছে। আসল payment gateway লাগবে না।

UI দেখতে সুন্দর হওয়া জরুরি নয়। জরুরি হলো ঠিকঠাক কাজ করা আর ব্যবহার সহজ হওয়া।

১.৪ API

> UI এই API-গুলোই call করবে। Contract হুবহু মানতে হবে, কারণ আমরা UI-এর পাশাপাশি সরাসরি API-এর উপরেও automated test চালাব। Response-এ বাড়তি field যোগ করতে পারেন, কিন্তু নিচের কোনো field বাদ দিতে বা তার নাম বদলাতে পারবেন না।

Base URL `http://localhost:8080`, সব request আর response JSON। শিক্ষার্থীর পরিচয় আসবে `X-Student-Id` header দিয়ে। এর মান হবে শিক্ষার্থীর ১৭ অঙ্কের জন্ম নিবন্ধন নম্বর। API-তেও প্রতিটা shift একটা আলাদা "class" (§১.৩ দেখুন)। Example-এর ID (`c_123`, `h_789`) শুধু উদাহরণ, ID-এর format আপনার পছন্দমতো।

#	Method ও Path	কাজ
1	`POST` `/classes`	নতুন class তৈরি
2	`GET` `/classes`	সব class-এর তালিকা, seat-এর হিসাব সহ
3	`GET` `/classes/{classId}`	একটা class-এর seat-এর অবস্থা
4	`POST` `/classes/{classId}/holds`	আবেদন (seat hold)
5	`GET` `/holds/{holdId}`	আবেদনের বর্তমান অবস্থা
6	`POST` `/holds/{holdId}/confirm`	ভর্তি ফি জমা, ভর্তি নিশ্চিত
7	`DELETE` `/holds/{holdId}`	আবেদন বাতিল

#	Method ও Path	কাজ
8	GET /me/holds	এই শিক্ষার্থীর সব আবেদন

1 POST /classes

```
// request
{ "name": "Class 6 - Morning Shift", "totalSeats": 120 }

// 201 response
{ "id": "c_123", "name": "Class 6 - Morning Shift", "totalSeats": 120 }
```

2 GET /classes

```
// 200 response
[
  { "id": "c_123", "name": "Class 6 - Morning Shift", "totalSeats": 120, "heldSeats": 15,
"confirmedSeats": 80, "availableSeats": 25 }
]
```

3 GET /classes/{classId}

```
// 200 response
{ "id": "c_123", "name": "Class 6 - Morning Shift", "totalSeats": 120, "heldSeats": 15,
"confirmedSeats": 80, "availableSeats": 25 }
```

4 POST /classes/{classId}/holds

HEADER `X-Student-Id`

```
// 201 response
{
  "holdId": "h_789",
  "classId": "c_123",
  "studentId": "20152691234567890",
  "status": "HELD",
  "expiresAt": "2026-11-01T04:10:00Z"
}
```

Seat না থাকলে `409`।

5 GET /holds/{holdId}

```
// 200 response
{ "holdId": "h_789", "classId": "c_123", "status": "HELD", "expiresAt": "2026-11-01T04:10:00Z" }
```

`status`-এর মান হতে পারে `HELD`, `CONFIRMED`, `EXPIRED` বা `CANCELLED`।

6 **POST** `/holds/{holdId}/confirm`

HEADER `X-Student-Id`

```
// 200 response
{ "holdId": "h_789", "status": "CONFIRMED" }
```

7 **DELETE** `/holds/{holdId}`

HEADER `X-Student-Id`। সফল হলে `204`।

8 **GET** `/me/holds`

HEADER `X-Student-Id`

```
// 200 response
[
  { "holdId": "h_789", "classId": "c_123", "className": "Class 6 - Morning Shift", "status":
"HELD", "expiresAt": "2026-11-01T04:10:00Z" }
]
```

১.৫ নিয়ম (বাধ্যতামূলক)

BACKEND

- কোনো অবস্থাতেই একটা class-এ `held + confirmed` সংখ্যা `totalSeats`-এর বেশি হবে না, যত request-ই একসাথে আসুক।
- আবেদনের মেয়াদ environment variable `HOLD_DURATION_SECONDS` দিয়ে ঠিক হবে (default `600`)। মেয়াদ শেষ হলে seat নিজে থেকে ফেরত যাবে, কাউকে কোনো API call করতে হবে না।
- একজন শিক্ষার্থী একটা class-এ একই সময়ে একটার বেশি active আবেদন বা নিশ্চিত ভর্তি রাখতে পারবে না।
- শুধু নিজের, মেয়াদ না পেরোনো `HELD` আবেদন confirm করা যাবে।
- একই আবেদনে confirm একাধিকবার এলেও `confirmedSeats` একবারই বাড়বে।
- App restart হলে কোনো data হারাবে না, `docker compose down` করে আবার `docker compose up` দিলেও।

UI

- সব কাজ UI থেকেই করা যাবে। কোথাও Postman বা সরাসরি DB লাগবে না।
- মেয়াদ শেষ হলে page reload ছাড়াই আবেদন `EXPIRED` দেখাবে, আর "ভর্তি ফি দিন" button বন্ধ হবে।
- Class-এর seat সংখ্যা page reload ছাড়াই কয়েক সেকেন্ড পরপর হালনাগাদ হবে।
- Page refresh করলে, বা browser বন্ধ করে আবার খুললেও, শিক্ষার্থী নিজের চলমান আবেদন দেখতে পাবে।
- যেকোনো error (seat শেষ, মেয়াদ শেষ, ভুল input, server বন্ধ) ব্যবহারকারীর বোঝার মতো ভাষায় দেখাবে। Blank page, বা শুধু console-এ error, চলবে না।

১.৬ Bonus (ঐচ্ছিক)

- একই database ব্যবহার করে backend-এর দুটো process আলাদা port-এ একসাথে চললেও `held + confirmed` সংখ্যা `totalSeats`-এর বেশি হবে না। কীভাবে নিশ্চিত করলেন, `ASSUMPTIONS.md`-এ লিখবেন।
- Frontend-এর automated test।

১.৭ যা জমা দিতে হবে

- একটা Git repository, commit history সহ, ভেতরে `backend/` আর `frontend/` folder। শেষে একটা বড় commit না দিয়ে কাজের সাথে সাথে commit করবেন।
- Backend আর frontend-এর `Dockerfile`, আর repository-র root-এ `docker-compose.yml` (§১.২ দেখুন)।
- `README.md`: Docker দিয়ে app চালানোর ধাপ, frontend কোন URL-এ খুলবে, আর test চালানোর নিয়ম।
- `ASSUMPTIONS.md` (ছোট): spec-এ যা স্পষ্ট বলা নেই সেখানে কী ধরে নিয়েছেন এবং কেন।
- Backend-এ অন্তত ৩টা automated test, তার একটা concurrency test। Concurrency test মানে অনেকগুলো request একসাথে পাঠিয়ে দেখানো যে seat সংখ্যার বেশি hold হচ্ছে না।
- `AI_USAGE.md` (ছোট): কোন AI tool, কোন অংশে ব্যবহার করেছেন, AI কোথায় ভুল করেছিল আর আপনি কীভাবে ধরলেন।

২ Project জমা দেওয়া

- নিজের GitHub account-এ একটা **public** repository বানান, আর পুরো কাজ সেখানে রাখুন।
- শেষ সময়ের আগে নিচের submission link দিয়ে repository-র link জমা দিন।
- শেষ সময়ে repository যে অবস্থায় থাকবে, আমরা সেটাই মূল্যায়ন করব। এর পরের কোনো commit বা push গোনা হবে না।
- Assignment-এর এই PDF public করবেন না, কারো সাথে share-ও করবেন না।

PROJECT জমা দিন

Project শেষ হলে নিচের link দিয়ে জমা দিন:

https://q-rp.com/c/4CXx

জমা দিতে কোনো সমস্যা হলে **hrd@onnorokom.com**-এ যোগাযোগ করুন।

ধন্যবাদ, শুভকামনা রইল!